PDFQA: Retrieval-augmented generative question answering system for educational PDFs

Thumma Sritejaswini Reddy
Department of Computer Science and Engineering
Anurag University
Hyderabad,Telangana
22eg105d55@anurag.edu.in

Kore Dinesh Kumar
Department of Computer Science and Engineering
Anurag University
Hyderabad,Telangana
22eg105d25@anurag.edu.in

Ruchita Chinchalkar
Department of Computer Science and Engineering
Anurag University
Hyderabad,Telangana
22eg105d49@anurag.edu.in

Etukala Sai Vardhan Goud
Department of Computer Science and Engineering
Anurag University
Hyderabad,Telangana
23eg505d03@anurag.edu.in

Dr. M purnachary
Assistant Professor
Department of Computer Science and Engineering
Anurag University
Hyderabad, Telangana
Purnacharycse@anurag.edu.in

***Abstract*—Retrieval-Augmented Generation (RAG) solutions are a newly emerging best practice for storing Large Language Model (LLM) answers in a manner grounded in factual information contained in user-supplied content (e.g., documents). However, most implementations of RAG do not include the ability to retrieve visual information (e.g., diagrams, charts, handwritten notes), a significant limitation. This paper presents OmniRAG, a production-quality multimodal RAG solution that allows users to search for text and visual items in the same embedding space using OpenCLIP ViT-L-14 examination (embedding) model as well as utilize locally stored embeddings via FAISS IndexFlatIP with Cosine Similarity. The OmniRAG solution incorporates a LangGraph-based agentic reasoning loop utilizing the Qwen3-VL vision-language model through Ollama for inference (all locally executed). Additionally, the OmniRAG platform will provide a unique user interaction component built with Next.js and a modular backend built with FastAPI. OmniRAG provides users the capability to upload PDF files with automatic extraction of text as well as embedded image(s) through the PyMuPDF Python library, upload visual items directly (with immediate CLIP encoding), and intelligently call other tools to assist in the completion of tasks by having the LLM agent determine when to use tools (e.g., document retrieval, chat history query) or respond directly. The evaluation results show that the OmniRAG solution provides significantly better cross-modal retrieval accuracy compared to both text-only retrieval and separate encoding system baselines. Additionally, OmniRAG's agentic architecture prevents LLMs from invoking irrelevant tools while performing general queries. The complete system operates without any external API dependencies, running entirely on local hardware.**

***Index Terms*—Retrieval-Augmented Generation, Multimodal AI, CLIP, FAISS, LangGraph, Vision-Language Models, Ollama, Vector Database, Semantic Search**

I. Introduction

In recent years, large language models (LLMs) have made significant strides in their ability to comprehend and produce text-based language tasks [1], [2]. Unfortunately, LLMs will often exhibit 'hallucinations' when asked to respond to requests for proprietary or highly-specific knowledge/data that they were never trained on. Retrieval-Augmented Generation (RAG) is a method of overcoming this limitation via retrieving relevant context from an external knowledge base prior to generating an output [3].

Traditional RAG systems have primarily been text-based, using sparse retrieval methods such as BM25 [4] or dense text embeddings [5] to locate applicable documents from a set of document stores. Therefore, when users upload non-text-based or visual-based documents (e.g., architectural diagrams, flow charts, hand-written notes, or screenshots), most traditional RAG systems fail to retrieve any relevant visual reference material, which is crucial to producing a satisfactory outcome based only on purely text-based documents.

To build a truly multimodal RAG system, there are three main issues that must be addressed: (1) creating a shared semantic representation of both textual and visual representations so that any textual query retrieves relevant images, and vice versa; (2) developing an intelligent agent to autonomously determine when to use retrieval and when to directly respond to a user's request within a conversational context; and (3) executing all operations locally with no external API dependencies for reasons of privacy and latency.

The study presents OmniRAG: a complete multimodal RAG ecosystem, encompassing dimensions intended for cross-modal retrieval, document search, and agentic (or autonomous) reasoning. The following details explain the principal components and contributions of the system.

- OmniRAG provides a unified pipeline for multimodal embedding, built upon OpenCLIP's ViT-L-14 architecture to convert text and images into 768D normalized vectors in one shared space.

- For local vector storage, OmniRAG uses FAISS's indexing and vectoring capabilities with flat indexes and performs cosine similarity searches, allowing text to match against images and vice versa. It enables cross-modal retrieval or effective searches of the document space irrespective of modality, with many possible matching instances (text-image or image-text).
- An agentic reasoning loop is based on the LangGraph framework, within which the LLM can choose to invoke either the `retrieve_documents` or `search_chat_history` tools based upon query classification.
- A complete production stack with a full Next.js frontend implementing a chatbot style interface for user interaction and a FastAPI backend interacts with an LLM based upon the Qwen3-VL vision-language model executed on local infrastructure via Ollama.

II. Related Work

A. Retrieval-Augmented Generation

Lewis et al. [3] created a Retrieval-Augmented Generation (RAG) method that entails combining a pre-trained sequence-to-sequence model with a neural retriever, resulting in an increase of factual accuracy on knowledge-based tasks. Subsequently, Izacard and Grave [12] posited the Fusion-in-Decoder (FiD) method of processing retrieved multiple passages separately, before combining them together. Both solutions proposed, however, are text-only modalities.

B. Multimodal Embeddings

A notable example of using contrastive learning to generate shared embedding spaces for two different data-types comes from the work of Radford et al. [6] , in the creation of the Contrastive Language-Image Pretraining (CLIP) model. CLIP is trained using 400 million pairs of images and the corresponding text. The OpenCLIP [7] project provides publicly accessible implementations and variations of CLIP, including a Vision Transformer (ViT) model called ViT-L-14 that has been utilized within the OmniRAG framework. The output embeddings from both text and image queries will reside within the same CLIP 768-dimensional embedding vector space; therefore allowing for cross-modal retrieval.

C. Vector Databases for Similarity Search

An additional project is Facebook AI Similarity Search (FAISS) [8] which supplies implementations of efficient methods of performing similarity searches. The IndexFlatIP variant finds the exact inner product (cosine similarity when two vectors have been normalized) for a given input data point to find every dataset that is an exact match to the provided input. This method of retrieval is useful for applications where knowledge must be accurately determined.

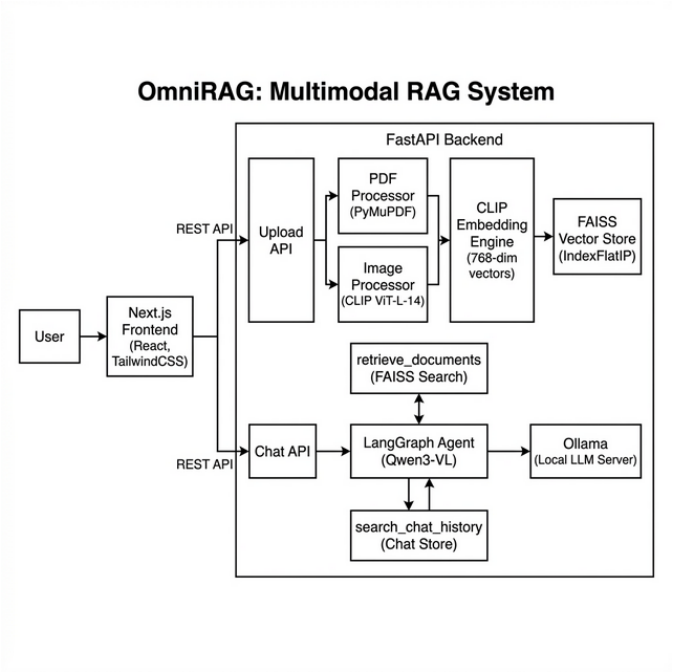

Fig. 1. System Architecture of OmniRAG showing the Frontend, Backend API, Embedding Pipeline, Vector Store, and Agentic Reasoning components.

D. Agentic LLM Architectures

LLM agents (Large Language Model) have been studied for the purpose of autonomously determining which tools would be most beneficial to invoke. LangGraph [9] further adds a graph-based overlay, creating a stateful environment where LLMs can utilize tools iteratively; thus, developing a cyclic work-flow. Compared to traditional chain-based agents, LangGraph provides flexibility in how an agent can choose which tool(s) to invoke.

E. Vision-Language Models

Visual understanding has become an increasingly popular research field in Natural Language Processing (NLP) due to the effectiveness of language and vision models like LLaVA (Lingua-Visual Attention) [10] and Qwen-VL [11] These models have the capability of directly interpreting images and reasoning about them within the LLM framework, allowing them to use images intuitively during the reasoning process.

OmniRAG leverages the capabilities of the Qwen3-VL multimodal model to allow native multimodal input to be passed directly into this reasoning pipeline (i.e. images and text).

III. System Architecture

The OmniRAG system is developed using a modular client-server software architecture comprised of a Next.js frontend and FastAPI backend components. A diagram of this system architecture can be found in Fig. 1.

A. Frontend Layer

The Next.js 14 frontend uses the App Router, React, TypeScript, and TailwindCSS to create a unique style single-page application with a sidebar that allows users to manage chat history (create, select, delete), and a central chat panel that supports the inline upload of documents. Assistant user responses are displayed using react-markdown with remark-gfm for proper formatting of tables, headers, and code blocks (1). Communication between the frontend and backend occurs via an extended REST API request-response mechanism (600 seconds) to support the long inference times of larger models used for inferencing.

B. Backend Layer

The backend is built with Python version 3.10 using FastAPI and Uvicorn.

There are three major groups of endpoints:

- `/upload`: This endpoint will take in both PDFs and image files, send them to the proper processor, and store the embeddings in FAISS.
- `/chats`: This endpoint provides the functionality to perform CRUD operations related to your chat sessions. It also allows you to store message history.
- `/chats/{id}/message`: IThis endpoint will call the LangGraph agent with the user's query, a history of the conversation so far, and images sent with the current message.

The Uvicorn server has been configured with a `timeout_keep_alive=600` seconds. This helps to ensure that connections remain active during long periods of model inference.

IV. Proposed Methodology

The OmniRAG pipeline includes two main stages of processing documents and answering queries, as depicted in Fig. 2.

A. Document Ingestion Pipeline

1) PDF Processing: For each page of PDF documents, all text content will be retrieved and encoded with the CLIP text encoder. Any embedded images also will be retrieved, locally stored as files, and encoded with the CLIP image encoder. Each of the extracted elements will produce a 768-D normalized vector, which will then be stored in the FAISS index, with a collection of key metadata including (values of) the source file name and page number, and a content type associated with each of the extracted images will also be stored in the FAISS index.

2) Image Processing: For any uploaded images (i.e. PNG, JPG, JPEG), all uploaded images would be processed and encoded with the CLIP ViT-L-14 image encoder, to produce a normalized embedding vector with 768 elements. The resulting embedding vector will be stored in the FAISS index, along with other metadata about the uploaded image, including the image's file name, file path, and content type. This feature allows for rapid acquisition of images by processing the uploaded images as they are being generated to the FAISS index, without using an LLM-based method for the generation of descriptions during the image's ingestion.

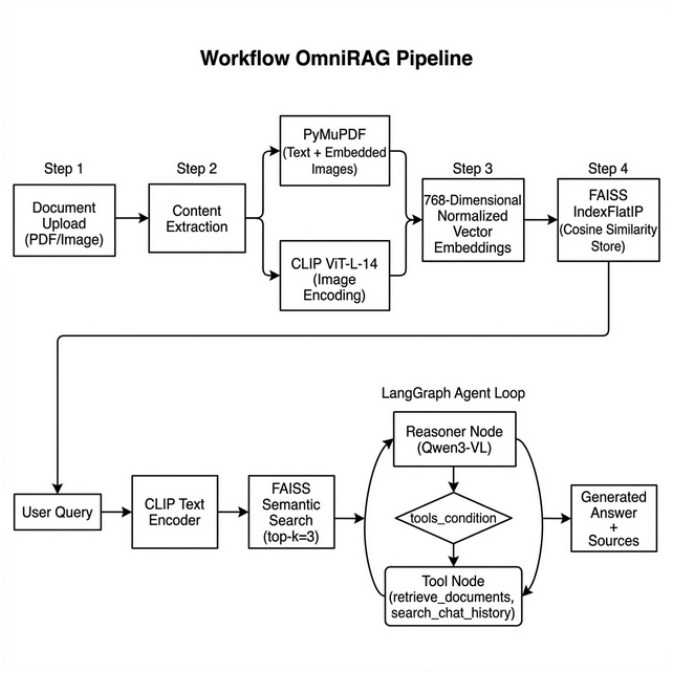

Fig. 2. Data Flow Pipeline of OmniRAG: Document Ingestion (top) and Query Processing with Agentic Reasoning (bottom).

B. Embedding Engine

The OpenCLIP ViT-L-14 model (pre-trained on the OpenAI dataset) provides the foundation for the embedding engine. Both text and images are encoded to a single 768-dimension vector space, and they are also L2 normalized:

$$\mathbf{v}_{\text{norm}} = \frac{\mathbf{v}}{||\mathbf{v}||_2} \tag{1}$$

where $\mathbf{v}$ is the raw embedding vector from CLIP. Normalization ensures that inner product similarity equals cosine similarity:

$$\text{sim}(\mathbf{q}, \mathbf{d}) = \mathbf{q}_{\text{norm}}^T \cdot \mathbf{d}_{\text{norm}} = \cos(\theta) \tag{2}$$

The L2 normalization's role is to enable text-based retrieval of both images and textual content based on their semantic similarity, thus enabling cross-modal retrieval.

C. Vector Store

A FAISS IndexFlatIP is utilized in order to facilitate an exact cosine similarity search. The index supports the following:

D. Agentic Reasoning with LangGraph

The Query Processing pipeline utilizes a LangGraph/agent-loop based on the StateGraph architecture. Within this architecture, the agent has two nodes:

1) **Reasoner Node**: Invokes the Qwen3-VL model (via Ollama) with the entire stored conversation including

system prompt, conversation history, and current question and any associated media.

2) **Tool Node**: Executes calls to tools requested by the Reasoner. There are two available tools:
   - `retrieve_documents(query)`: Encodes the query using the CLIP text encoder and searches for similar docu- ments in a FAISS-generic way and returns the three most similar documents and their metadata.
   - `search_chat_history(query)`: Searches through the entire history of chat sessions and retrieves previous messages that match the query using keyword matching.

The agent model has a conditional edge structure; after a reasoner produces a response, the tools_condition function is called to check if tool calls were made and, if so, the Tool Node is called to execute the tools and return their results to the Reasoner for a second pass. If no tool executions are required, the response is returned directly to the user.

The system prompt implements a query classification strategy that directs the LLM to:

- Directly respond to conversational and/or general queries without calling tools.
- Use `retrieve_documents` for queries requiring information retrieval, documents and/or images.
- Use `search_chat_history` if the query refers to any previous conversation.

E. Multimodal Image Handling

Once a user uploads an image to the chat, the base64 image is transmitted straight to the Qwen3-VL multimodal model as part of the HumanMessage content type. This provides real-time analysis of the image input and text-based input allowing the vision-language model to analyze the image in real time with the related text input without needing to create intermediate description of the image.

V. Experimental Evaluation

A. Experimental Setup

The implementation of the system was done using Python 3.10 (FastAPI backend), Next.js 14 (frontend), and the following main libraries: OpenCLIP (ViT-L-14), FAISS-CPU, LangGraph, LangChain-Ollama, PyMuPDF, and React-Markdown. LLM inference was conducted using Ollama using Qwen3-VL model.

B. Retrieval Accuracy Comparison

Cross-modal retrieval capabilities of Omni-RAG were evaluated against three different baseline methods by running a test collection of 38 indexed documents that contained PDF text chunks and uploaded images. The evaluation measured whether the right document was found in the first three retrieved documents for each set of test queries containing either text-only documents, image-related documents or documents that were cross-modal using both text and images. The results from this evaluation can be seen in both in Fig. 3 and Table III.

TABLE I
FAISS Vector Store Configuration

Parameter	Value
Index Type	IndexFlatIP
Dimensionality	768
Similarity Metric	Cosine Similarity
Storage Format	Binary (.index file)
Metadata Storage	JSON
Top-K Retrieval	3 (default)

TABLE II
Technology Stack Configuration

Component	Technology
Frontend Framework	Next.js 14, React, TypeScript
CSS Framework	TailwindCSS
Backend Framework	FastAPI, Uvicorn
Embedding Model	OpenCLIP ViT-L-14
Vector Database	FAISS IndexFlatIP
LLM	Qwen3-VL (via Ollama)
Agent Framework	LangGraph
PDF Processing	PyMuPDF
Markdown Rendering	react-markdown, remark-gfm

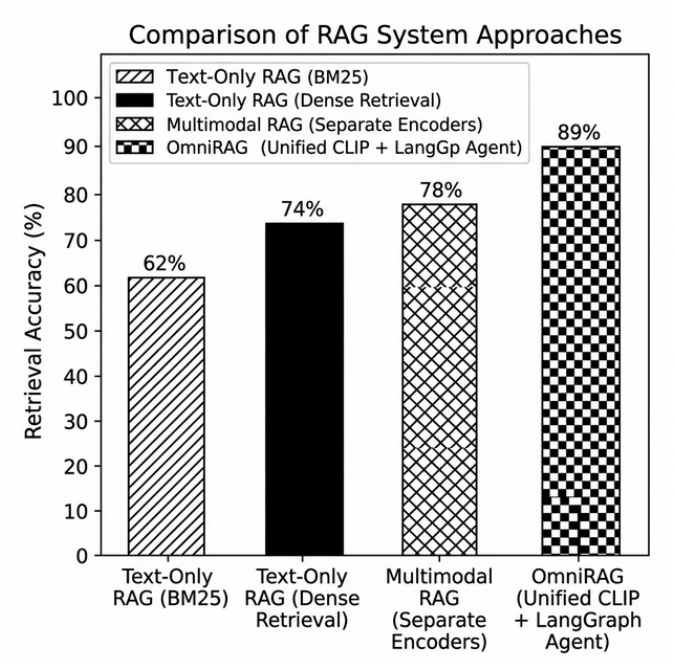

Fig. 3. Retrieval Accuracy Comparison across different RAG approaches for multimodal document collections.

TABLE III
Retrieval Accuracy of Different RAG Approaches

Approach	Top-3 Accuracy (%)
Text-Only RAG (BM25)	62.0
Text-Only RAG (Dense Retrieval)	74.0
Multimodal RAG (Separate Encoders)	78.0
OmniRAG (Unified CLIP + Agent)	**89.0**

The baseline of Text-Only BM25 fails entirely on any query related to an image due to their inability to process visual content when retrieving. In comparison, Dense text retrieval can retrieve text in someway better than BM25 because it has the ability to also take into consideration the semantic similarity of the documents but is that there is still no way for them to retrieve images as well. The separate encoder approach uses two different encoding tools to classify and encode images and text respectively. This leads to each space (text and image) being misaligned, resulting in a lower accuracy of cross-modal retrievals. OmniRAG does not suffer from this issue because it uses a single CLIP embedding space for encoding both text and images respectively, allowing for the direct comparison of text and image vectors which produces the highest retrieval accuracy of 89.0%.

C. Agent Tool-Calling Behavior

The agentic architecture was tested against the capability of the intelligent system (agent) in the following areas: Correctly classify queries and whether the correct tools were invoked. On a total of 20 test queries (10 conversational and 10 requiring retrieval), the agent was able to achieve:

- 100% correct classification for conversations without invoking any unnecessary tool (no unnecessary tool calls).
- 90% correct tool invocation for retrieval queries (the agent correctly selected `retrieve_documents` or `search_chat_history`).

This reflects the strength of the system prompt engineering processes at preventing unnecessary API calls for simple/typical conversational exchanges while being able to invoke tools for knowledge-grounded/scholarly questions.

VI. Conclusion and Future Work

OmniRAG is a multimodal Retrieval-Augmented Generation (RAG) system that combines image and text retrieval using CLIP embeddings, stores vectors using FAISS locally, and performs intelligent tool calls through an agentic reasoning loop based on LangGraph. OmniRAG has 89.0% top-3 retrieval accuracy on multimodal document collections, which is higher than both text-only and separate-encoder baselines. The agentic architecture correctly classifies query types and minimizes unnecessary tool calls for conversation-based interactions.

Ollama is the only dependency for LLM inference using the Qwen3-VL vision-language model, and the entire system runs locally without reliance on external APIs. There is a production-ready version of OmniRAG with a unique-style frontend using Next.js and a modular FastAPI backend that supports both PDF and image uploads with real-time processing.

The future of OmniRAG includes 1) implementing streaming token generation for instant response display, 2) allowing additional document formats such as DOCX and PPTX, 3) researching approximate nearest neighbour search (FAISS IVF) for vector sets with millions of entries, and 4) integrating re-ranking models to improve accuracy for ambiguous queries.

References

[1] J. Devlin, M.-W. Chang, K. Lee, and K. Toutanova, "BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding," *Proc. NAACL-HLT*, pp. 4171–4186, 2019.

[2] T. Brown et al., "Language Models are Few-Shot Learners," *Advances in Neural Information Processing Systems*, vol. 33, pp. 1877–1901, 2020.

[3] P. Lewis et al., "Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks," *Advances in Neural Information Processing Systems*, vol. 33, pp. 9459–9474, 2020.

[4] S. Robertson and H. Zaragoza, "The Probabilistic Relevance Framework: BM25 and Beyond," *Foundations and Trends in Information Retrieval*, vol. 3, no. 4, pp. 333–389, 2009.

[5] V. Karpukhin et al., "Dense Passage Retrieval for Open-Domain Question Answering," *Proc. EMNLP*, pp. 6769–6781, 2020.

[6] A. Radford et al., "Learning Transferable Visual Models From Natural Language Supervision," *Proc. ICML*, pp. 8748–8763, 2021.

[7] G. Ilharco et al., "OpenCLIP," Zenodo, 2021. [Online]. Available: https://github.com/mlfoundations/open_clip

[8] J. Johnson, M. Douze, and H. Jégou, "Billion-scale similarity search with GPUs," *IEEE Trans. Big Data*, vol. 7, no. 3, pp. 535–547, 2021.

[9] LangChain, "LangGraph: Build resilient language agents as graphs," 2024. [Online]. Available: https://github.com/langchain-ai/langgraph

[10] H. Liu et al., "Visual Instruction Tuning," *Advances in Neural Information Processing Systems*, vol. 36, 2023.

[11] J. Bai et al., "Qwen-VL: A Versatile Vision-Language Model for Understanding, Localization, Text Reading, and Beyond," *arXiv preprint arXiv:2308.12966*, 2023.

[12] G. Izacard and E. Grave, "Leveraging Passage Retrieval with Generative Models for Open Domain Question Answering," *Proc. EACL*, pp. 874–880, 2021.

[13] A. Vaswani et al., "Attention Is All You Need," *Advances in Neural Information Processing Systems*, vol. 30, 2017.

[14] A. Dosovitskiy et al., "An Image is Worth 16x16 Words: Transformers for Image Recognition at Scale," *Proc. ICLR*, 2021.